

Library Management System

1. Justification

The design pattern that we have chosen is Strategy.

The Strategy Pattern is appropriate for the Library Management System because it allows different search methods — such as searching by title, author, ISBN, or genre — to be changed dynamically at runtime without modifying the main search logic. Instead of using large conditional statements, each search algorithm is placed in its own strategy class implementing the SearchStrategy interface. This design follows important SOLID principles: it supports the Open/Closed Principle by allowing new search types to be added without changing existing code, ensures Single Responsibility by giving each strategy one task, supports Liskov Substitution because any strategy can replace another, and follows Dependency Inversion since the context depends on abstractions rather than concrete classes.

The SearchContext class provides runtime flexibility through the setStrategy() method, allowing users to switch search methods instantly without recreating objects or reloading the system. This directly reflects the Strategy Pattern concept explained in Lecture 9, where algorithms are extracted into separate classes and delegated by a context object. Compared to using inheritance or the Template Method pattern, Strategy is more flexible because it replaces rigid inheritance hierarchies with composition, making the system easier to extend and maintain.

2. UML Class Diagram

